Project 3 – Sprint 5 Planning Document

Sprint 5 Review Date: Friday May 15th
Scrum Master for Sprint 4: Sathkrith Segu

Section 1 – Retrospective Rule Application

- Sprint 4 Structural Issue:
  Every Sprint Goal component met its task-level Definition of Done, but the full set of features was never exercised as one continuous demo before demo day. Individual tests confirmed each task worked however nothing tested them together. Integration friction, persistence verification time, and visual ambiguity in the CO2 chart came up during the demo.
- Copy your rule for Sprint 5 from the Sprint 4 Change Plan (exact wording):
  At the final 10 minutes of every class period beginning Dev Day 3, Yohaan (Demo Verifier) will run an unbroken end-to-end walkthrough of every completed Sprint Goal component on a clean, logged-out browser, and any feature that fails, requires verbal explanation, or behaves inconsistently must be moved back to “In Progress” on the GitHub board before class ends.
- When does this rule happen during a normal class period?
  (Be specific: start, during work time, end of class, etc.)
  This will be enforced 5 minutes after class every day. That way all of us have a designated time we can always meet up.
- What would I SEE happening if this rule is followed correctly?
  You will see Yohaan project his screen, open a fresh incognito tab, log in as a test account, and run through every completed Sprint Goal component back-to-back without pausing (upload a file, run a CO2 calc, log a ride, open the calendar, etc). Any feature that requires verbal explanation, fails, or behaves inconsistently is flagged on the GitHub board in real time during the walkthrough.
- Where will this show up in your workflow? (select all that apply)
  ☐ Task creation
  ☐ Task completion
  ☒ Testing
  ☐ Board updates
  ☐ Pull requests
  ☐ Daily Scrum
- What is ONE way this rule could still fail in Sprint 5?
  One way this rule could still fail in Sprint 5 is that a developer might write a boundary test case into the Definition of Done during planning time but then skip it during completion are pressed for time, marking it Done without demonstrating the edge case. The tester (Yohaan) must actively ask "show me what happens with an invalid input" rather than passively accepting the happy-path demo.
- What happens immediately if this rule is NOT followed?
  Every task moved to Done since the last completed walkthrough is automatically

reverted to "In Progress" on the GitHub board. No new task may move to Done at the next class until the missed walkthrough is run.

- Who is responsible for enforcing this rule?
  Yohaan
- If this person is absent, who enforces the rule?
  Aarush
- What changes (if anything):
  Nothing changes (Aarush does everything that Yohaan would have done)

Section 2 – Sprint Goal (Expansion + Stability)
Your goal must add new value WITHOUT breaking your existing system.

- Sprint Goal (By the end of Sprint 5, a user will be able to…):
  By the end of Sprint 5, a user will be able to view and filter available bike trails by difficulty and surface type; see the money savings of choosing to bike over other transportation modes; and log rides to their activity log with a correctly stored date and no ability to log rides in the future.
- What existing system MUST remain stable?
  The Supabase backend (user authentication, rides table, gallery storage, CO2 calculator routing, Leaflet map, and announcements page) must remain stable.
- What would failure look like at demo?
  The demo would fail if trail cards do not render or filters do not work; the money savings figure is missing or incorrect; or the activity log date bug reappears (wrong date stored or future dates accepted).

Section 3 – Scope Control (Hard Constraint)

- What IS included in this Sprint (2 – 4 items MAX):

Backlog Item	Why it supports the goal
Money Savings Display	The Money Calculations helps complete Momentum's core value proposition: users will be able to see both the environmental AND financial benefits of biking.
Trail Browsing	The Trail Browsing feature gives users a reason to open Momentum before a ride, and not just after: browsable, filterable trails will directly help encouraging biking.
Click or tap here to enter text.	Click or tap here to enter text.
Click or tap here to enter text.	Click or tap here to enter text.

- What is NOT included in this Sprint (REQUIRED)
  Personalized route recommendations, ride sharing with the community, social/public profile features, and displaying energy statistics are NOT included in this Sprint.
- What are you intentionally delaying and why?
  We are intentionally delaying personalized route recommendations because that feature would require some form of ML based on user history data that does not yet exist. Adding this feature now would introduce a new data pipeline and significant algorithmic complexity on top of the three already-committed features, making failure of this demo likely.
- Why is this amount of work realistic for your team this Sprint? Make sure you take into account absences due to AP Exams.
  The activity log fix is 3 simple, targeted tasks on existing code. The money savings display extends existing Carbon Savings Calculator without new backend work. Trail browsing follows the same Supabase-fetch-and-render pattern the team has used for locations and the gallery. With 11 tasks distributed across 4 team members and all tasks scoped to one class period or less, the load for this sprint is pretty realistic.

Section 4 – Scope Validation
Do NOT list new backlog items. Use only the items from section 3.

- Which item is the highest risk to your Sprint Goal? Why?
  The item that is the highest risk to our Sprint Goal is Trail Browsing because it requires creating a new Supabase table, seeding data, building a card-based UI, implementing client-side filtering, and adding a detail view — four distinct tasks that must be done different owners.
- Which item improves stability or user experience?
  The activity log fix directly improves stability and user experience because it resolves two concrete bugs that were visible during the Sprint 4 demo. A ride tracker that stores the wrong date is not a usable product.
- If you had to remove ONE item by Dev Day 3, which would it be? Why?
  If we had to remove ONE item by Dev Day 3, it would be the Trail Browsing because while it is extremely valuable to our goal, it is the newest and most complicated of the three features and the most likely to cause demo failure.
- Why is this total scope realistic for your team?
  This total scope is realistic for our team because each backlog item has been decomposed into tasks of one class period or less with clear owners and testable completion criteria (including boundary tests). With 11 tasks across 4 team members, the load is balanced.

Section 5 – Task System (Strict Execution)
Tasks MUST be ≤ 1 class period.
If the definition of done is vague, the task is invalid.

Add additional rows to this table as necessary by copy and pasting the contents of an existing row and then modifying.

Task	Owner	Backlog Item	Done = (clear criteria)	Includes Testing?	How is it tested?
Fix date-offset bug in LogRide.tsx: replace new Date() with new Date(Date.UTC(...)) to ensure the stored date matches the user's local calendar date, not UTC-shifted.	Sriram	Activity log fix	Submitting a ride at any time of day stores the exact local date selected; no + or - 1 day shift visible in Supabase rides table.	Yes	Submit a ride at 11 PM local time; confirm Supabase row shows today's local date, not tomorrow.
Add date validation to LogRide.tsx: disable the date <input> so that max={new Date().toISOString().split('T')[0]} is enforced, preventing future dates from being selectable or submittable.	Aarush	Activity Log fix	Done = the date picker offers no date beyond today; attempting to submit a future date via DevTools manipulation returns a validation error.	Yes	Try selecting tomorrow in the picker - confirm it is greyed out. Manually set a future date via DevTools and submit, confirm form rejects it.
Regression test: log 3 rides on different dates (past, today) using two test accounts; confirm correct dates in Supabase, correct calendar highlighting, and no cross-user data leakage.	Yohaan	Activity Log fix	Done = all 3 rides stored with correct dates; calendar highlights correct days; user B cannot see user A's rides.	Yes	Log rides as user A and user B; inspect Supabase rides table and confirm dates; verify calendar for each user independently.

Extend Calculator.tsx: add a 'Money Saved' output section below the CO2 result. Using the distance and mode inputs already present, compute savings = (selected_mode_cost_per_mile - bike_cost_per_mile) x distance. Display as 'You saved $X.XX by biking instead of [mode].'	Aarush	Money Savings Display	Done = entering 10 miles with Car selected displays a dollar savings figure with 2 decimal places and no console errors.	Yes	Input 10 miles, select Car — verify output matches manually computed value. Repeat for Transit. Verify bike vs. walking shows $0.00 or a clear message.
Add a money savings bar to the existing visual comparison section in Calculator.tsx: render a second row of bars (below CO2 bars) showing dollar cost for bike vs. selected mode, labeled with mode name and dollar amount.	Sriram	Money Savings Display	Done = two money bars render side-by-side with correct proportional heights and dollar labels across at least 3 different distance/mode combinations.	Yes	Test with 5 mi Car, 20 mi Transit, 1 mi Walking - confirm bars resize proportionally and labels update correctly each time.
Style and integrate money savings section with Tailwind CSS to match existing Calculator page color scheme; confirm layout has no overflow on desktop viewport.	Sathkrith	Money Savings Display	Done = /calculator loads with money savings section visible, no layout overflow, styling matches existing app on desktop viewport.	Yes	Navigate to /calculator on desktop — inspect for overflow; compare color scheme against another app page side-by-side.

Create a 'trails' table in Supabase: columns id (uuid), name (text), difficulty (text), length_miles (float8), surface (text), region (text), description (text). Insert 6 seed rows covering varied difficulty and region values.	Yohaan	Trail Browsing	Done = 6 seed rows visible in Supabase dashboard under the trails table with all columns populated.	Yes	Verify visually in Supabase table editor; confirm all 6 rows have no null required fields.
Build TrailList.tsx: fetch all trails using supabase.from('trails').select('*') and render each as a card showing name, difficulty badge, length, and surface. Add filter dropdowns for difficulty (Easy/Moderate/Hard) and surface (Paved/Gravel/Dirt).	Sriram	Trail Browsing	Done = all 6 seed trails render as cards; selecting 'Easy' in difficulty filter shows only Easy trails; filter combinations work without page reload.	Yes	Select each difficulty filter value and each surface value independently; confirm correct subset of trails appears each time.
Add TrailList to React Router at '/trails'. Add navigation link in app's existing nav bar. Style page with Tailwind CSS to match app color scheme.	Sathkrith	Trail Browsing	Done = /trails loads through nav link with no layout overflow; styling matches existing app on desktop viewport.	Yes	Navigate to /trails via nav bar on desktop; confirm no overflow or styling mismatch.
Add a trail detail view: clicking a trail card opens a modal or detail panel showing the full description and all fields. Clicking outside or pressing Escape closes it.	Aarush	Trail Browsing	Done = clicking any trail card opens the detail view with correct data; Escape and outside-click both close it with	Yes	Click 3 different trail cards; verify correct data loads each time; test Escape and outside-click to close.

			no console errors.		

- What is the most critical task on this list?
  The most critical task on this list is fixing the date offset bug in our activity log. This task is absolutely important because it represents something that was not completed in our previous sprint, requiring its explicit mention in our sprint goal. If we do not complete this task, it may promote further snowplowing and impacted tasks in the sprint and future work.
- Fully describe how that task will be handed off if necessary.
  If Sriram is unable to complete the task, it will be handed off to Yohaan. If further aid is required, we will, as a group, debug the issue, walking through it line by line to figure out what may be the problem.
- What must already exist for handoff to work?
  For the handoff to work, a clear plan must already exist regarding each individual's tasks that fully meets and prioritizes the sprint goal. The task must also be on the project board and transparency must be in place for us to understand the situation with the task and make sure it is completed for the sprint.

Section 6 – Testing Integration

- When during the task is testing performed?
  Testing is going to be performed directly after the task is completed, before it is moved to Done on the GitHub board. Each task's definition of done includes ideal test cases and boundary ones with possible failure and edge cases as well. Testing will not be post-poned to the end of the sprint.
- Who verifies the task meets Definition of Done BEFORE it moves to Done?
  Yohaan Ardeni
- What happens immediately if a test fails?
  The task will stay In Progress until the test case for both the ideal situation and the edge cases pass thoroughly. For tasks that are assigned to our designated tester Yohaan Ardeni, they will be tested by Aarush Kapoor.
- If the developer is absent, who verifies testing?
  Aarush Kapoor

Section 7 – Workflow Discipline

- When EXACTLY is the board updated each class?
  (Example: last 3 minutes of class before dismissal)
  The board is updated twice per class period: once at the start of class (within the first 5 to 10 minutes) during Sathkrith's sprint goal coverage check, and once in the final 5 minutes of class when Aarush reviews task progress and moves cards that have met their Definition of Done.
- Who checks that the board reflects reality?
  Sriram

- How do tasks move (To Do -> In Progress -> Done)?
  A task moves from To Do to In Progress the moment the assigned developer makes their first commit toward it. A task moves from In Progress to Done only after the developer has demonstrated testing of both normal and edge cases.
- Rule to prevent last-minute work completion:
  Before the Daily Scrum each day, Sathkrith will have a 1–2 minute personal check-in with each developer to ask how their current task is progressing and whether they believe it is manageable within a single period. If a developer is stuck or unlikely to finish, the team addresses it during the Daily Scrum rather than waiting.
- How will the board stay accurate if members are absent?
  The board will stay accurate as our Scrum Master will ensure constant communication throughout the entirety of an absence. As most of these absences are due to AP exams, developers will still be able to communicate after and before and make sure that we will have progress that reflects daily results. An example of this communication would be daily discussion in our group chats, with large milestone recorded both on the portal and our personal discussions page.
- Who ensures updates still happen?
  Aarush

Section 8 – Team Availability Plan

Complete the following table. Enter a new row for each date a specific team member will be absent. In the coverage plan describe EXACTLY what work continues during that class period.

Team Member	Absent Dates	Impacted Tasks	Coverage Plan
Sathkrith	5/7, 5/11, 5/13, 5/14	TrailList	Async double coding to ensure that if one person isn't present, another can cover
Yohaan	5/11, 5/13, 5/14	Trails Backend	Delegation worst case will fall to Aarush or Sathkrith
Aarush	5/8, 5/13, 5/14	Trail Detail	Delegation worst case will fall to Sathkrith or Sriram
Sriram	5/13, 5/14	TrailList	Async double coding to ensure that if one person isn't present, another can cover

- How does work continue when members are absent?
  We use asynchronous check ins daily in our group chat on other platforms to ensure that all tasks are completed in a timely manner and we would essentially mimic the in-class work time with a call

- How do responsibilities shift when members are absent?
  When members are absent, responsibilities don't change because we are adaptable as developers and students, and our team's development plan accounts for member absences.

Section 9 – Failure Prevention Plan

- Early warning sign that your Sprint is failing:
  A task remains In Progress for more than one class period without a corresponding commit on GitHub or a sprint goal component has no task card on the board at the start of a development day, or a task is marked Done without a documented test result are what we need to realize that we are behind. Oftentimes it's the testing that leaves us with last second bugs.
- Trigger condition (REQUIRED FORMAT):
  If [X] is not complete by [time], we will [action]
  If the Activity Log Fix tasks are not both marked Done by the end of Dev Day 2, sathkrith will flag it at the next scrum and the team will prioritize completing the fix before beginning any new trail sectioning tasks so that we don't snowpile our work.
- Who is responsible for making this adjustment?
  Sathkrith
- What will you REMOVE from scope if needed?
  The trail detail view modal will be removed first. This is because the card list with filters does the browsing and the cutting itself. If further cuts are needed, the money savings visual comparison bars will be removed, leaving only the dollar output alongside the existing Co2 text output.
  Click or tap here to enter text.

Section 10 – Expansion Justification

- Why is this the right feature to build now?
  The money savings display and trail browsing are the right features to build now because we need to finish the MVP: users can find a trail, bike it, log the ride, and immediately see both the environmental and financial benefit of their choice. Without money savings, the calculator only gives the user values that they cant use. Without trail browsing, users have no in-app reason to plan a ride. In its core, Momentum is a app that lets the users get access to more biking friendly ideas and routes. The previous checkpoint was more focused on the green house gasses data, this sprint will balance it out with the idea of biking.
- How does it improve user experience or product value?
  The money savings figure makes the benefit of biking concrete and personal as users can see what they saved on a specific trip, which is a stronger motivator than CO2 valyes alone for many users (from a sample of users of various ages). Trail browsing directs

Momentum from a logging tool into a planning tool, giving it a reason to be opened before a ride rather than just logging afterwards.

- Why is your system ready to handle this expansion?
  (Must reference a specific system improvement from your Sprint 3 retrospective)
  Our supabase authentication (Sprint 2), per-user gallery and activity log data model (Sprints 3 and 4), and the CO2 calculator routing and component structure (Sprint 4) are all stable and tested. The trail browsing feature can follow the exact same Supabase-fetch-and-render pattern used for the map locations table in Sprint 3, which was completed fully and on time. The money savings display extends existing Calculator.tsx code that has already been reviewed and works.

Section 11 – Final Commitment Checklist

☒ Our rule is observable and enforceable
☒ All tasks meet the ≤1 class period requirement.
☒ Every task includes testing
☒ Scope is realistic and constrained
☒ Board update expectations are clear
☒ We have a plan to detect failure early